ICT162

Proctored Online Exam - January Semester 2025

Object Oriented Programming

Thursday, 24 April 2025 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **5** questions.
2. Answer all parts of each question in the corresponding answer box.
3. All answers must be typed in the answer box and submitted in Examena.
4. This is a **Closed-book** examination.
5. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.
6. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.
7. **One (1)** A4-sized (double-sided) hardcopy Cheat Sheet allowed.

At the end of the examination.
To end the examination, please click on the "Submit and end exam" button to submit your answers.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT162 Copyright © 2025 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Analyse the Python classes below.

```python
from abc import ABC, abstractmethod

class A(ABC):
    def runJobs(self):
        return self.job1() + "\n" + self.job2()

    def job1(self):
        return "A running job 1"

    @abstractmethod
    def job2(self):
        pass

class B(A):
    def job2(self):
        return "B running job 2"

class C(B):
    def job1(self):
        return "C running job 1"
```

Carefully analyse each part below and assume that each part is independent.

- State whether there is an error in the Python statement(s).
- If there is an error, specify which statement causes that error and explain why the error occurs.
- If there is no error, explain what the statement(s) does (e.g., which class object is created) and state the output produced by the statements.

```python
# Q1a (i)
b = B()
print( b.runJobs() )

# Q1a (ii)
c = C()
print( c.runJobs() )

# Q1a (iii)
a = A()
print( a.runJobs() )
```

(8 marks)

Question 2

Analyse the Python classes below. What is the output of the main() function? (There is no error in the codes)

```python
class Parent:
    def __init__(self, x: int):
       self.__cost = x

    @property
    def cost(self):
        return self.__cost

    def specialty(self):
        return "Chendol"

    def method1(self):
       return "{} ${}".format(self.specialty(), self.cost)

class Child(Parent):
    def specialty(self):
        return "Durian " + super().specialty()

class GrandChild(Child):
    def method1(self):
       return "Ice Cream {} ${}".format(self.specialty(), self.cost)

class GreatGrandChild(GrandChild):
    def specialty(self):
        return "Chocolate " + super().specialty()

def main():

    # Q1b (i)
    p = Parent(3)
    print( p.method1() )

    # Q1b (ii)
    c = Child(6)
    print( c.method1() )

    # Q1b (iii)
    gc = GrandChild(8)
    print( gc.method1() )

    # Q1b (iv)
    ggc = GreatGrandChild(10)
    print( ggc.method1() )
```

(12 marks)

Question 3

CapitaStar is CapitaLand's multi-mall, multi-store cardless rewards programme that gives member benefits as they shop, dine and enjoy. Members get to accumulate STAR$ with purchases from participating outlets across CapitaLand Malls in your city and use them to redeem vouchers and/or other promotional item.

The CapitaCard is a multi-mall loyalty credit card that offers multiple rewards and benefits. Cardholder automatically enrolled into CapitaStar programme and can accumulate up to 6X STAR$ (capped at S$1,200 per calendar month) for every eligible purchase at participating CapitaLand Malls.

The class diagram in Figure Q3 is simplified for this exam. Study the diagram.

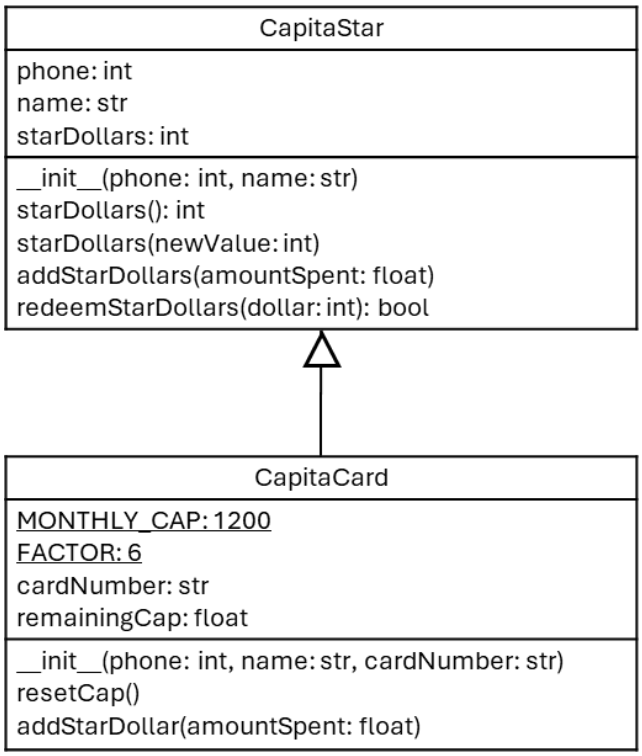


Figure Q3

Question 3a

Implement the class – CapitaStar.

- It has THREE instance variables:
  - phone (int): contact number for this CapitaStar member
  - name (str): name of the member
  - starDollars (int): accumulated STAR$ for this member
- Constructor initialises the two instance variables, phone and name, using the given parameters.
  - starDollars is defaulted to 0
- Getter and setter methods for starDollars. Use the property/setter decorator.
- The addStarDollars() method has amountSpent (float) as parameter. The method must validate the following before computing and adding the STAR$ for this member:
  - To qualify for STAR$, the amountSpent must be more than $10
  - If qualified, every $1 spend will earn the member 1 STAR$. For example, $21.60 will earn the member 21 STAR$.
- A redeemStarDollars() method with amount (int) as parameter. If there are enough STAR$, subtract the amount and return True, otherwise return False.

(10 marks)

Question 3b

Write Python class definitions for CapitaCard, which is a subclass of CapitaStar

- Two class variables:
  - _FACTOR: represent the 6X STAR$ privilege for CapitaCard holder
  - _MONTHLY_CAP: defines the $1,200 cap per calendar month
- It has TWO additional instance variables:
  - cardNumber (str): card number like "1234-4444-2300-1766"
  - remainingCap (float): the remaining cap for this month to earn 6X STAR$
- Apart from phone and name, the constructor initialises ONE additional instance variable: cardNumber (str) and set the remainingCap to the _MONTHLY_CAP.
- Implement the method resetCap() such that the remainingCap is updated to the value of the class variable _MONTHLY_CAP.
- Given the amountSpent (float), the implementation of the addStarDollars() method will be as follows:
  - To qualify for STAR$, the amountSpent must be more than $10
  - If the remainingCap is already 0, every $1 spend will earn the member 1 STAR$.
  - If the amountSpent is within the remainingCap, every $1 spend will earn the member 6 STAR$. For example, $10.80 will earn the member 60 STAR$. The remainingCap will be deducted with the amountSpent
  - If the amountSpent is more than the remainingCap, only the remainingCap amount will earn the 6X STAR$, the rest of amountSpent will earn 1 STAR$ per $1 spent. The remainingCap will become 0 after this.

(16 marks)

Question 3c

Based on your Python codes in Q2b for class CapitaCard, explain what type of overriding you have implemented for method *addStarDollars()*.

(4 marks)

Question 4

Study the class diagram in Figure Q4. You are to develop Python classes that raise exceptions. Please note that the class diagram is simplified for exam purposes.

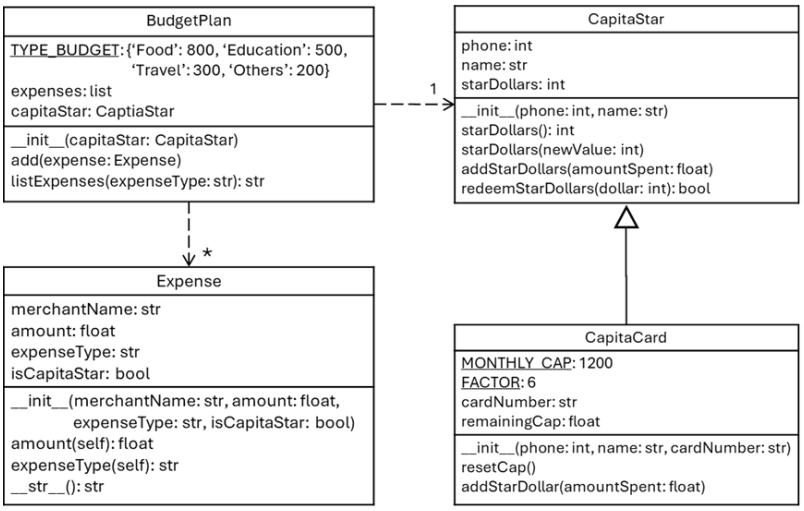


Figure Q4

Question 4a

Define the class ExpenseException, which is a subclass of the Exception class. This class has no additional attribute or method.

(1 mark)

Question 4b

Implement the Expense class.

The Expense class has:

- FOUR instance variables:
  - merchantName (str): name of the merchant/outlet for this spending.
  - amount (float): representing the amount spent.
  - expenseType (str): the type of expenses (food, education, travel & others).
  - isCapitaStar (bool): indicating if this spending is on a participating outlet across CapitaLand Malls.
- Constructor performs the following:
  - Check if the parameter expenseType contains a valid value in the class variable TYPE_BUDGET of the BudgetPlan class. If not, raise ExpenseException with appropriate message.
  - Initialise these 4 instance variables with the parameter values – merchantName, amount, expenseType and isCapitaStar.
- Getter method for instance variables, amount, expenseType and isCapitaStar. Use the property decorator.
- The __str__ method returns a string representation of an Expense object. These are examples in the suggested format:

```
Merchant: InnisFries    Amount: $23.40    Type: Food
Merchant: MathsAcer    Amount: $90.00    Type: Education
```

(8 marks)

Question 4c

Implement the class BudgetPlan with the following specifications:

- ONE class variable TYPE_BUDGET: a dictionary containing the allocated budget for each expense type. The key is the expense type, while value contains the allocated budget.
- TWO instance variables:
  - expenses (list): collection of Expense objects.
  - capitaStar: contain the CapitaStar object if owner of this budget plan is a member. Default is None.
- Constructor will initialise instance variable capitaStar with the given parameter and set instance variable expenses to an empty list.
- The add() method will add the given Expense object into the collection (list). However, the method must:
  - check if this additional expense will cause the total spending of this expense type to be more than its allocated budget. If yes, raise ExpenseException with appropriate message.
  - If capitaStar is not None, meaning this budget plan owner is a capitaStar member, invoke the appropriate method such that STAR$ earned by this expense is credited into the capitaStar.

  - The listExpenses() method returns a string representation of Expense objects that matches the given expenseType. For example, expenses for type 'Food':

```
Merchant: InnisFries    Amount: $23.40    Type: Food
Merchant: Ya Kun    Amount: $6.20    Type: Food
Merchant: Din Tai Fung    Amount: $76.80    Type:
Food
```

  - If expenseType does not match any of the pre-defined expense type in class variable TYPE_BUDGET, raise ExpenseException with appropriate message.
  - If the expenseType is not given or None, this method will return the string representation of all Expense objects, and the current STAR$ if applicable. For example:

```
Merchant: InnisFries    Amount: $23.40    Type: Food
Merchant: MathsAcer    Amount: $90.00    Type:
Education
Merchant: Ya Kun    Amount: $6.20    Type: Food
Merchant: Din Tai Fung    Amount: $76.80    Type:
Food
STAR$: 99
```

(21 marks)

Question 5

Question 5 has 2 parts: Q5a & Q5b. Answer all 2 parts of this question.

Question 5a

Implement **TWO (2)** class methods in class BudgetPlan as illustrated below in Figure Q5a.

BudgetPlan
TYPE_BUDGET: {'Food': 800, 'Education': 500, 'Travel': 300, 'Others': 200} expenses: list capitaStar: CaptiaStar
__init__(capitaStar: CapitaStar) updateBudgetByType(type: str, budget float) listBudget(): str add(expense: Expense) listExpenses(expenseType: str): str

Figure Q5a

- Class method updateBudgetByType() has type and budget as parameters. The method will:
  - add the given type (key) and budget (value) into the class variable TYPE_BUDGET, if the type is not found in the dictionary, or
  - if the given type is found in the class variable TYPE_BUDGET, update the budget accordingly.
- Class method listBudget() returns a formatted string showing all the types and budgets found in class variable TYPE_BUDGET.

```
Food        $800.00
Education   $500.00
Travel      $300.00
Others      $200.00
```

Provide the python codes for the 2 class methods only.

(6 marks)

Question 5b

A partially developed GUI is showcased below in Figure Q5b, to allow user to check, add/update the budget type and amount.

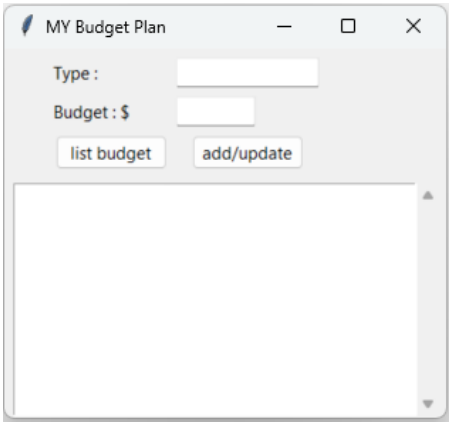


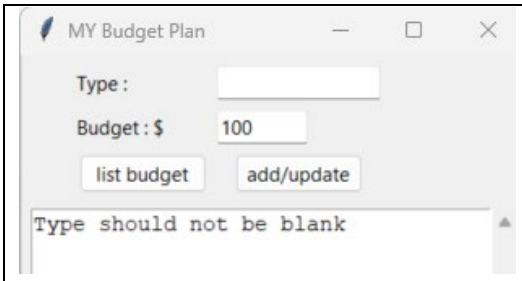
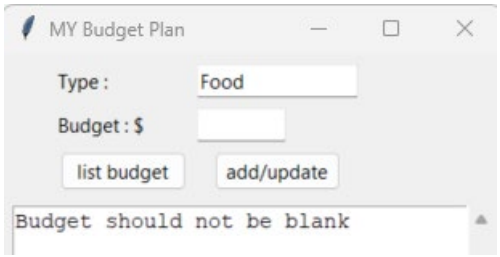
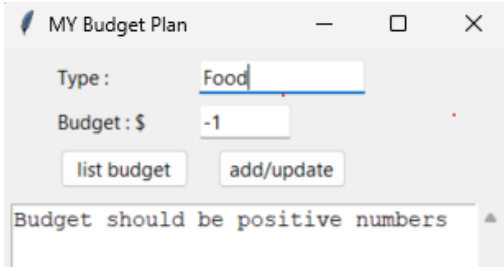
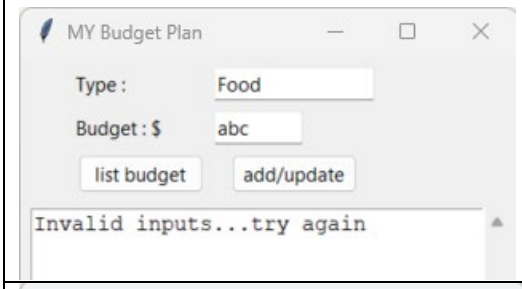
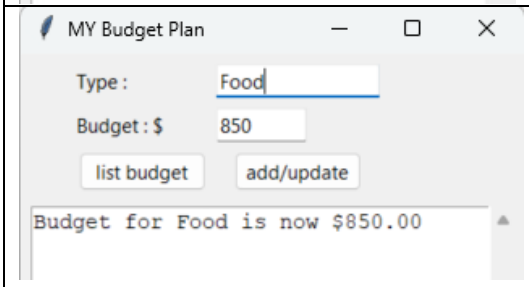
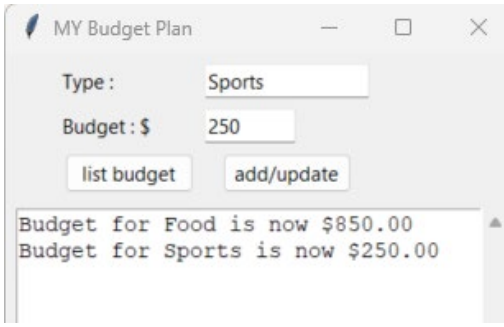
Figure Q5b

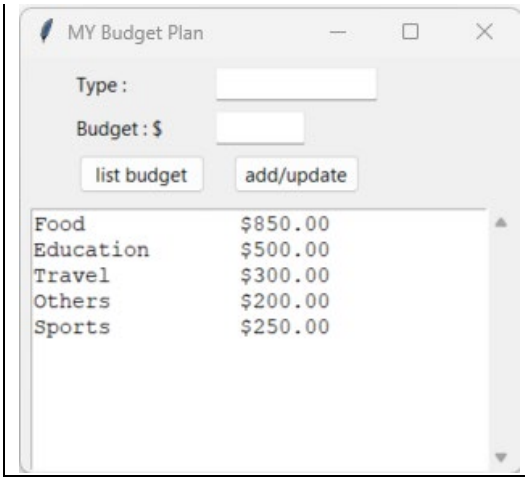
The following is the initial code for the GUI shown in Figure Q5b. In line 4, we assumed the class BudgetPlan is inside the file Q5a.py

```
1  import tkinter as tk
2  from tkinter import ttk
3  from tkinter import scrolledtext
4  from Q5a import BudgetPlan
5
6  class BudgetGUI:
7      def __init__(self):
8          self.__win = tk.Tk()
9          self.__win.resizable(True, True)
10         self.__win.title("MY Budget Plan")
11         self.create_widgets()
12         self.__win.mainloop()
13
14     def create_widgets(self):
15         dataFrame = ttk.Frame(self.__win)
16         dataFrame.grid(row=0, column=0, padx=8, pady=4)
17
18         type_lbl = ttk.Label(dataFrame, text="Type : ")
19         type_lbl.grid(row=0, column=0, sticky='W')
20         self.__type = tk.StringVar()
21         self.__type_Ety = ttk.Entry(dataFrame, width=15,
22                                     textvariable=self.__type)
23         self.__type_Ety.grid(row=0, column=1, sticky='W', padx=4, pady=4)
24
25         budget_lbl = ttk.Label(dataFrame, text="Budget : $")
26         budget_lbl.grid(row=1, column=0, sticky='W')
27         self.__budget = tk.StringVar()
28         self.__budget_Ety = ttk.Entry(dataFrame, width=8,
29                                       textvariable=self.__budget)
30         self.__budget_Ety.grid(row=1, column=1, sticky='W', padx=4, pady=4)
31
32         self.__list_btn = ttk.Button(dataFrame, text="list budget",
33                                      command=self.list)
34         self.__list_btn.grid(row=2, column=0, padx=4, pady=4)
35         self.__update_btn = ttk.Button(dataFrame, text="add/update",
36                                        command=self.update)
37         self.__update_btn.grid(row=2, column=1, padx=4, pady=4)
38
39         outputFrame = ttk.Frame(self.__win)
40         outputFrame.grid(row=1, column=0, padx=8, pady=4, columnspan=2)
41         self.__scrol_stxt = scrolledtext.ScrolledText(outputFrame, width=35,
42                                                       height=10, wrap=tk.WORD)
43         self.__scrol_stxt.grid(row=0, column=0, sticky='WE')
44         self.__scrol_stxt.config(state=tk.DISABLED)
45         self.__type_Ety.focus()
46
47     def list(self):
48         pass
49
50     def update(self):
51         pass
```

Write statements that will implement the event handler for “list budget” and “add/update” buttons, in other words, complete the 2 callback functions – list() and update().

Below are demonstrations of the BudgetGUI:

   	Ensure the callback function perform the following validations: • Type cannot be blank • Budget cannot be blank • Budget must be > 0 • Budget must be numeric
 	If both type and budget are filled with proper values, invoke the appropriate class method to add/update the type and budget into BudgetPlan.

	If the “list budget” button is clicked, invoke the appropriate class method to obtain budget information and showcase in the scrolled text area.

(14 marks)

----- END OF PAPER -----